



第六章 差分隐私

目 录

「01」

知识点一：概念和性质

「02」

知识点二：拉普拉斯机制

「03」

知识点三：差分隐私实践

「04」

知识点四：指数机制

「05」

知识点五：随机响应机制

引语

- 差分隐私概念最早由Cynthia Dwork⁷⁴等人于2006年提出，区别于以往的 k -匿名等隐私保护方案，其主要贡献是给出了**对个人隐私泄露的数学定义，可以在最大化查询结果可用性的同时，保证单个用户隐私泄露不超过预先设定的 ϵ 。差分隐私并不是要求保证数据集整体性的隐私，而是对数据集中的每个个体的隐私提供保护。**
- 我们在前序的章节中已经讨论过各类匿名化处理数据的方法，也对数据的脱敏进行了充分的讨论。然而，这些算法虽然能一定程度上保护隐私，但是在某些特定的攻击方法下，其仍然会暴露个体的隐私，差分攻击就是其中的一种。

1. 差分攻击

□ 表展示了一个由学生的姓名、性别和成绩三个属性组成的数据表。设姓名和性别是公开数据，可以响应任意针对其的查询，而我们打算保护个体成绩的隐私。因此，**要求查询服务器只响应对非个体的（如，所有女生）的成绩查询请求。**

姓名	性别	成绩
Aisha	F	Fail
Benny	M	Pass
Erica	F	Fail
Fabio	M	Fail
Johan	M	Fail
Ming	M	Pass
Orhan	M	Pass

查询 “Ming” 的成绩：不响应

查询 “有多少学生通过了考试”：响应

有没有个体信息泄露？无

1. 差分攻击

姓名	性别	成绩
Aisha	F	Pass
Benny	M	Pass
Erica	F	Fail
Fabio	M	Fail
Johan	M	Fail
Ming	M	Pass
Orhan	M	Pass

查询“有多少女生通过了考试”：响应

有没有个体信息泄露？无

在这种情况下，我们可以利用差分攻击来达到获取个体隐私的目的

查询“所有学生中通过考试的人数”：4

查询“除了Aisha外所有学生通过考试的人数”：3

差分攻击的核心是通过寻找两个仅相差一条记录的数据集，对其分别做同样查询，再比较返回结果的差异，从而获取两个集合所相差的记录的敏感信息

2. 差分隐私

- 为了抵御差分攻击，最直观的想法是**通过对查询结果加入一定的扰动**（即反馈的查询结果并非真实的结果），**使得查询结果不再精确**，使得攻击者在进行差分攻击时获得的查询结果无法用于区分只差一条记录的两个集合。

姓名	是否患癌症
Tom	是
Bob	否
Amy	是
Lily	否
Alice	是

设一个基本的查询函数 $f(i) = \text{count}(i)$ ，
来查询数据集D**前i条数据中罹患癌症的人数的**

只需执行查询 $f(5)$ 和 $f(4)$ ，通过 $f(5) - f(4)$
即可知晓Alice是否罹患癌症，形成差分攻击

2. 差分隐私

姓名	是否患癌症
Tom	是
Bob	否
Amy	是
Lily	否
Alice	是

为了抵御差分攻击，可以在查询结果上增加一个随机数，使得攻击者每次得到的结果都是含有一个随机值的，以此来扰乱攻击者得到的查询结果。这个随机数称之为噪音(noise)，其一般是遵循某种分布产生的随机数。在添加噪音后，我们可以得到新的 $f'(i) = count(i) + noise$ 。

- 假设我们产生的 $noise \in [-1,1]$ ，结合数据集，此时 $f'(5) = [2,4], f'(4) = [1,3]$ ，且由于添加的噪音是遵循某种分布均匀随机的，故两次查询得到的结果也是在两个值域上均匀随机的，攻击者将无法再通过对两次查询的差值来获取Alice是否罹患癌症的隐私信息，达到了保护个体隐私的目的，即能够提供差分隐私。我们称能提供了随机化输出的算法 $f'(x)$ 为一个满足差分隐私的算法。

2. 差分隐私

定义 $x \in X$ 为域 X 中的元素，从 X 中抽取 n 个 x 的集合组成数据集 D ，其中属性的个数为维度 d 。 X^n 表示数据集 D 的集合，即从 X 中抽取 n 个 x 组成的数据集的集合。若两个数据集 D 和 D' 具有相同的属性结构，二者之间只有一条记录不同，则称 D 和 D' 为**相邻数据集** (Neighboring Datasets)。设算法 M 为一个随机算法， R 为实数域。

定义5.1 差分隐私。 若随机算法 $M: X^n \rightarrow Y$ ，对于任意子集 $S \subseteq Y$ ，在任意两个相邻数据集 $D, D' \in X^n$ 上，满足 $\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S] + \delta$ ，则称随机算法 M 满足 (ϵ, δ) -差分隐私。

参数 δ 即表示随机算法 M 不满足差分隐私的概率， $\delta \in [0, 1)$ 。当 $\delta = 0$ 时，称 M 满足 ϵ -差分隐私，又称为纯差分隐私，而 (ϵ, δ) -差分隐私又称为近似差分隐私。

2. 差分隐私

- 在 ϵ -差分隐私中, $e^{-\epsilon} \leq \frac{\Pr[M(D) \in S]}{\Pr[M(D') \in S]} \leq e^{\epsilon}$, 这表示算法 M 在两个相邻数据集上输出相同结果的概率比值在 $[e^{-\epsilon}, e^{\epsilon}]$ 之间, 也就是说, **隐私参数 ϵ 可以用来控制算法 M 在两个相邻数据集上输出相同结果的概率比值, 体现了差分隐私的保护水平。**
- $\epsilon \geq 0$, ϵ 越小表示隐私保护水平越高, 相反 ϵ 数值越大表示隐私损失越大。
- 当 $\epsilon = 0$ 时隐私保护水平达到最高, 意味着对于任意相邻数据集, 算法将输出两个概率分布完全相同的结果。**一般来讲, ϵ 在小于1的情况下能提供较高的隐私保障。**
- ϵ 的取值也反映了数据的可用性, ϵ 越小, 数据可用性越低。

2. 差分隐私

- 从另一方面来看， ϵ 的取值也反映了数据的可用性：在普通情况下， ϵ 越小，数据可用性越低。 ϵ 越大，隐私保护越弱，但数据可用性越高。
- 在一些场景中，必须在 ϵ 取值较大的情况下，才能实现有效的数据分析或模型训练任务。
- 因此，隐私预算 ϵ 的取值需要结合实际场景和需求，在输出结果的隐私性和可用性之间进行权衡。

有关差分隐私，说法正确的是

- A ϵ 越小，数据可用性越好
- B 差分隐私机制可以保证数据集整体性的隐私
- C ϵ 越小，数据隐保护程度越高
- D 必须在 ϵ 较小的情况下，才能实现有效的数据分析或模型训练任务

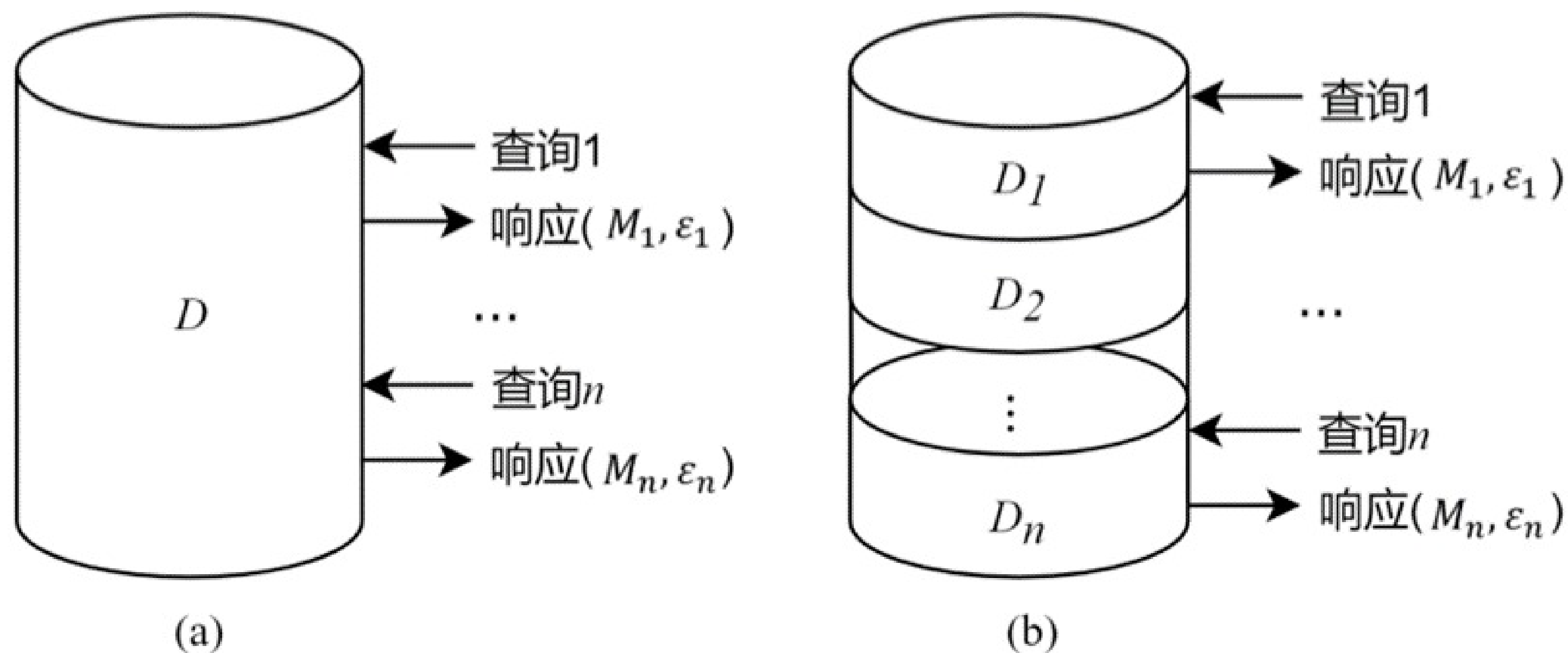
3. 差分隐私性质

定理5.1 后处理不变性. 若随机算法 $M: X^n \rightarrow Z$ 满足 (ϵ, δ) -差分隐私, 对于任意随机映射 $f: Z \rightarrow Z'$, 则 $f \circ M: X^n \rightarrow Z'$ 是 (ϵ, δ) -差分隐私的。

差分隐私保证: 如果数据分析者除了数据分析任务之外不能对数据集进行额外查询, 就无法增加数据集中每条记录的隐私损失。也就是说, 如果我们使用随机性算法保护了个人的隐私, 那么数据分析者就不能仅通过背景知识以及算法的输出来增加隐私损失, 无论是在正式定义中, 还是在任何直观意义上。形式化来说, 与数据集无关的映射 f 与一个满足 (ϵ, δ) -差分隐私的算法 M 组合起来, 仍然满足 (ϵ, δ) -差分隐私, 我们将其称为后处理不变性 (Post-Processing)。

3. 差分隐私性质

在现实的应用场景中，往往需要多次对数据集进行查询来满足日常的数据分析任务。在这种情况下，差分隐私的串行组合性(Sequential Composition)和并行组合性(Parallel Composition)提供了在多次使用随机算法查询数据集时计算隐私损失的方法。



差分隐私的组合性质：(a)串行组合性，(b)并行组合性

3. 差分隐私性质

定理5.2 串行组合性. 给定一组差分隐私算法 $M_1, M_2, \dots, M_n$, 每个 M_i 满足 (ϵ_i, δ_i) -差分隐私。利用这组机制对相同的数据集 D 进行查询, 组合算法 $M(D) = (M_1(D), M_2(D), \dots, M_n(D))$ 满足 $(\sum \epsilon_i, \sum \delta_i)$ -差分隐私。

串行组合性表明用一组差分隐私算法查询同一个数据集后, **数据集中每条记录的隐私损失将不超过全部所有算法导致的隐私损失的总和**。然而, 将随机算法的隐私损失简单求和的方式往往并不尽如人意, 因为这样会导致最终的隐私损失中更为重要的参数 ϵ 偏大。

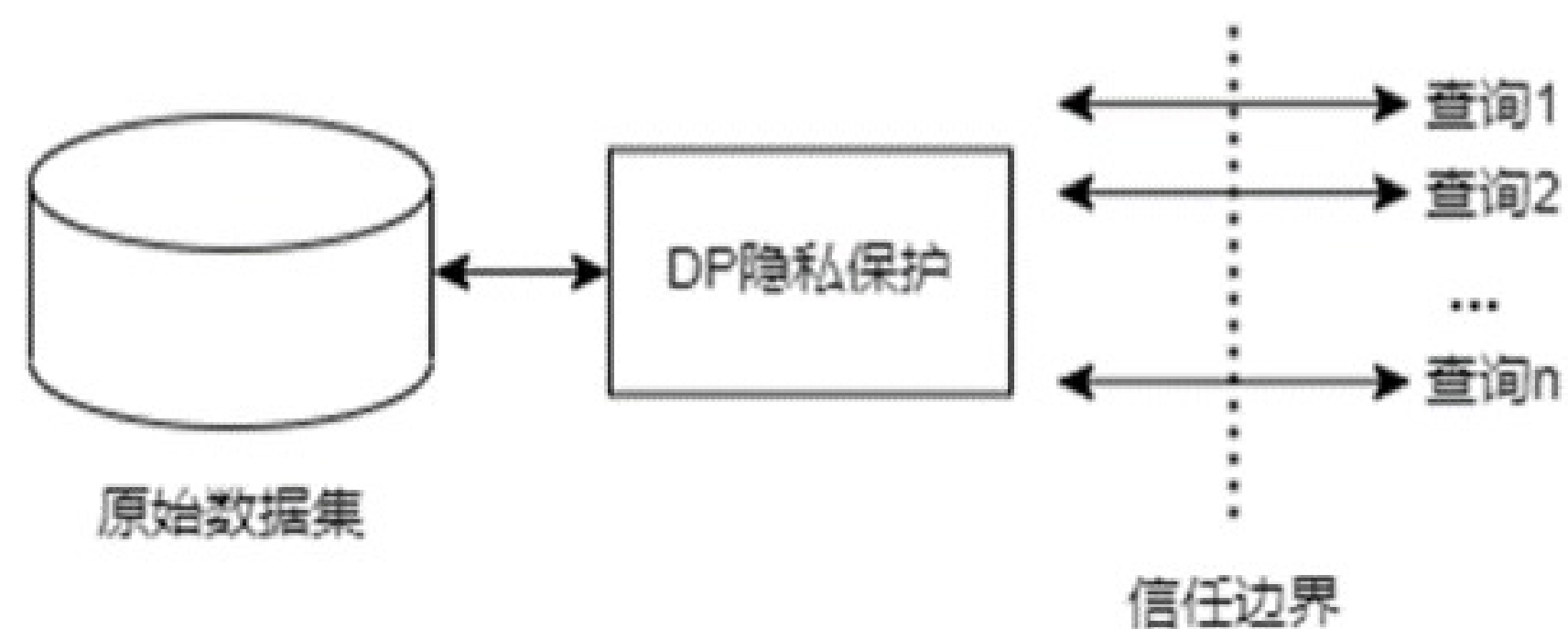
3. 差分隐私性质

定理5.3 并行组合性. 给定一组差分隐私机制 $M_1, M_2, \dots, M_n$, 每个 M_i 提供 (ϵ_i, δ_i) -差分隐私保证。若集合 D 包含 n 个不相交的子集 $D = D_1 \cup D_2 \dots \cup D_n$, 将这组算法分别作用在这些子集上, 构成的组合算法 $M(D) = (M_1(D_1), M_2(D_2), \dots, M_n(D_n))$ 会提供 $(\max \epsilon_i, \max \delta_i)$ -差分隐私。

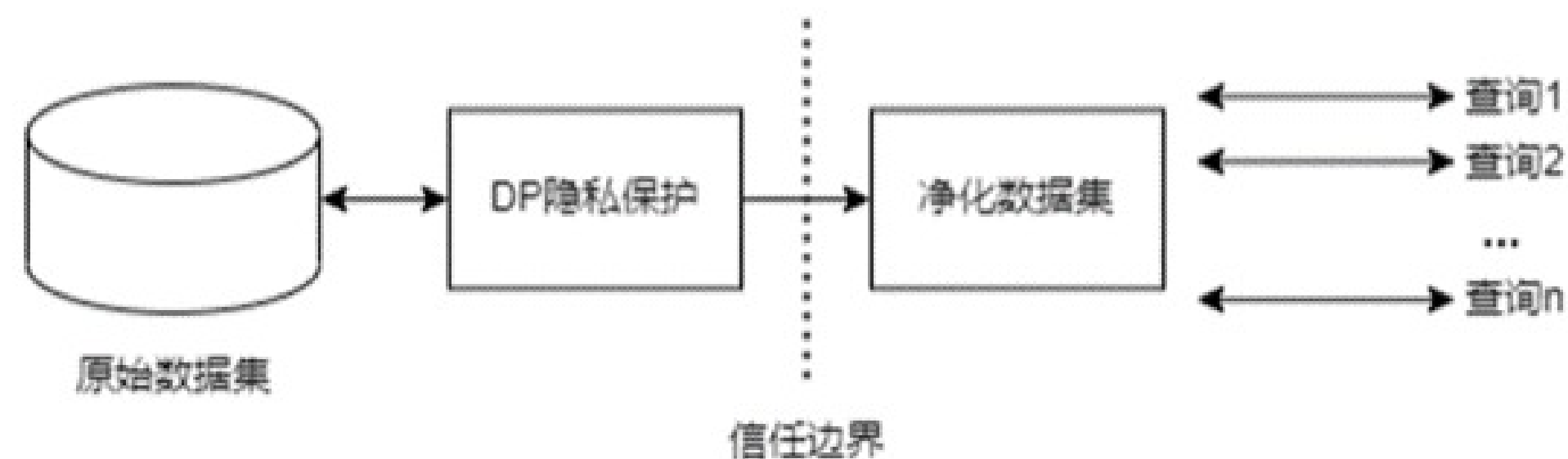
相比于串行组合性, 差分隐私的并行组合性更容易理解。由于差分隐私度量的是数据集中每一条记录的隐私损失, 当随机算法不查询该数据集时, 数据集中每条记录不损失隐私。在并行组合性中, 每条记录只有在随机算法查询该数据集时损失隐私, 差分隐私又是度量最差情况下隐私损失的定义, 因此并行组合性即对应所有算法中隐私损失最差的算法。

4. 差分隐私模型

根据对于多次查询的响应方法不同，差分隐私的发布模型分为交互式和非交互式两种。



(a)交互式数据发布



(b)非交互式数据发布

在交互式数据发布中，**每次查询都要消耗隐私预算**，逐个回答查询集 F 中的查询，直到耗尽全部隐私预算

非交互式只需在发布合成数据集时消耗隐私，由于差分隐私的**后处理不变性**，对合成数据集的后续查询任务，不会进一步泄露原始数据集的隐私。

有关差分隐私，说法错误的是

- A 非交互式数据发布，后续查询不会消耗隐私预算
- B 差分隐私的并行组合特性，是指数据集中每条记录的隐私损失将不超过全部所有算法导致的隐私损失的总和
- C 与数据集无关的映射 f 与一个满足 (ϵ, δ) -差分隐私的算法 M 组合起来，仍然满足 (ϵ, δ) -差分隐私

目 录

「01」

知识点一：概念和性质

「02」

知识点二：拉普拉斯机制

「03」

知识点三：差分隐私实践

「04」

知识点四：指数机制

「05」

知识点五：随机响应机制

1. 拉普拉斯机制

```
SELECT COUNT(*) FROM D WHERE Sick = “糖尿病”
```

这是一个典型的计数查询，显然会受到差分攻击的影响。对于数值型的查询结果，拉普拉斯机制是在输出的数值上直接添加噪音。不过，在加入噪音之前，我们也要考虑另一个因素的影响。考虑在当前数据集 D 只改动一条数据形成的相邻数据集 D' ，对于当前这个查询而言，如果我们只改动一条数据的情况下，那前述的SQL查询的结果改动至多为1。

但是，如果我们将查询变成这样：

```
SELECT 3*COUNT(*) FROM D WHERE Sick = “糖尿病”
```

改动一条数据的情况下，这个查询的结果可能会改变3而不是1了。为了解决类似的问题，**差分隐私引入了灵敏度的概念，其度量当数据集有一条改动的情况下，对查询结果的最大改变。**

1. 拉普拉斯机制

定义5.2 全局灵敏度. 设有查询函数 $f: X^n \rightarrow R^d$, 输入为一个数据集, 输出为 d 维实数向量。对于任意相邻数据集 D 和 $D' \in X^n$, 则

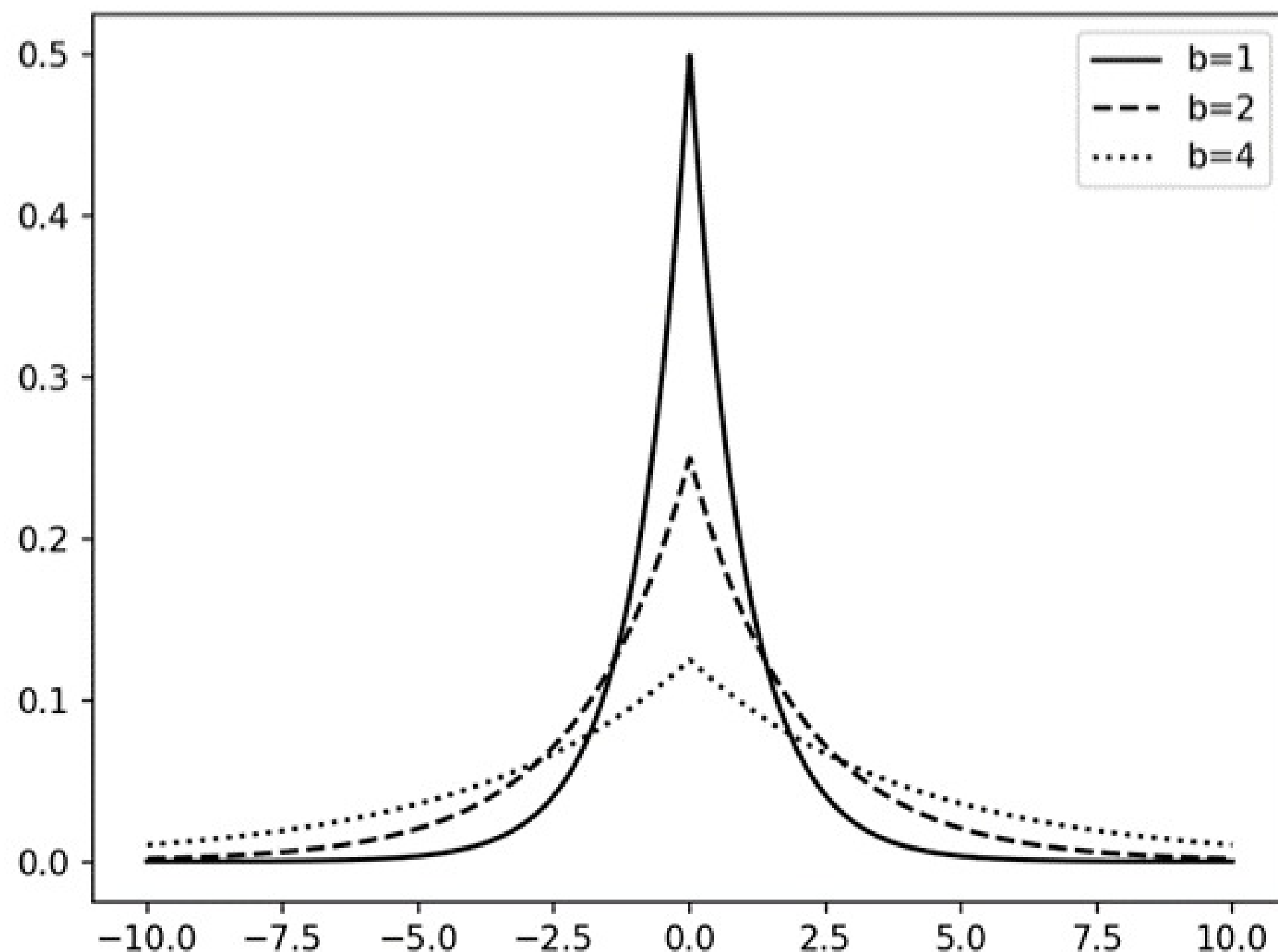
$$GS_f = \max_{D, D'} ||f(D) - f(D')||_p.$$

称为函数 f 的全局灵敏度。其中 $||f(D) - f(D')||_p$ 是 $f(D)$ 和 $f(D')$ 之间的 p -阶范数距离, 记为 l_p 灵敏度。

对于不同的机制, 灵敏度的范数也不同, 拉普拉斯机制使用1阶范数距离 (即距离之差的绝对值), 而高斯机制采用二阶范数 (即欧几里得距离), 具体取决于对应机制的隐私分析方法。

1. 拉普拉斯机制

拉普拉斯分布：拉普拉斯机制通过向查询结果或原始数据加入服从拉普拉斯分布的噪声来实现差分隐私。那么，为什么会选择拉普拉斯分布呢？



横轴表示随机变量 x ，纵轴表示相对应的概率密度（具体纵轴输出的是 x 出现的概率大小）， b 是尺度参数。如图所示，拉普拉斯分布是对称无偏的。这个特点使得随机化算法的噪声是无偏的（即数学期望是0），进而使得噪声的均值为0，即使得水平得以大大降低，能提高数据有效性。

1. 拉普拉斯机制

我们使用 $Lap(b)$ 来表示位置参数为0、尺度参数为 b 的拉普拉斯分布， $Lap(x|b)$ 表示在尺度参数为 b 的时候输出结果为 x 的概率，即概率密度函数，表示为：

$$Lap(x|b) = \frac{1}{2b} \exp\left(-\frac{|x|}{b}\right).$$

根据差分隐私的理论，加入的噪声参数满足 b 为 $\frac{1}{\epsilon}$ ，即能满足 ϵ -差分隐私。

1. 拉普拉斯机制

定理5.4 拉普拉斯机制 设函数 $f: X^n \rightarrow R$, $\Delta f = \max_{D, D'} ||f(D) - f(D')||_1$

为 l_1 灵敏度。给定数据集 $D \in X^n$, 则随机算法

$$M(D) = f(D) + Y, \quad Y \sim \text{Lap}(\Delta f / \epsilon)$$

能提供 ϵ -差分隐私保护。其中, X^n 是一 n 维的输出域, Y 为独立的、服从

尺度参数 b 为 $\Delta f / \epsilon$ 的Laplace分布的随机变量。

2. 拉普拉斯噪声

拉普拉斯噪声： 基于产生随机变量的组合方法来产生这个噪音。

```
/*  
函数功能：利用混合同余法产生（a,b）区间上均匀分布  
的随机数  
输入参数说明：  
a给定区间的下限  
b给定区间的上限  
seed长整型指针变量， *seed 为伪随机数的种子  
*/  
double uniform_data(double a, double b, long int *  
seed) {  
    double t;  
    *seed = 2045.0 * (*seed) + 1;  
    *seed = *seed - (*seed / 1048576) * 1048576;  
    t = (*seed) / 1048576.0;  
    t = a + (b - a) * t;  
    return t;  
}
```

以隐私参数 β 和随机种子 $seed$ 生成拉普拉斯分布随机数→

```
/*  
函数功能：求解laplace分布概率累积的反函数，利用该  
反函数产生laplace分布的随机数  
输入参数说明：  
beta拉普拉斯分布参数  
seed 长整型指针变量， *seed 为伪随机数的种子  
*/  
double laplace_data(double beta, long int *  
seed) {  
    double u1, u2, x;  
    u1 = uniform_data(0.0, 1.0, seed);  
    u2 = uniform_data(0.0, 1.0, seed);  
    if (u1 < 0.5)  
    {  
        x = beta * (log(2*u1)+u2);  
    }  
    else  
    {  
        x = u2 - (beta * log(2*(1-u1)));  
    }  
    return x;  
}
```


目 录

「01」

知识点一：概念和性质

「02」

知识点二：拉普拉斯机制

「03」

知识点三：差分隐私实践

「04」

知识点四：指数机制

「05」

知识点五：随机响应机制

统计查询应用

实验： 对一个数据集（zoo.csv）进行统计查询，该数据集描述了一个动物园喂食的场景，第一列中数据为动物名称，第二列中数据为动物每天消耗的胡萝卜数量。查询定义为“每日进食超过55根胡萝卜的动物数量”。请设计相关的隐私保护方案，确保查询过程不泄露信息。

	A	B
1	Aardvark	1
2	Albatross	88
3	Alligator	35
4	Alpaca	99
5	Ant	69
6	Anteater	14
7	Antelope	77

方案一：交互式发布

对抗重复推测攻击

重复攻击

- 重复攻击是针对差分隐私的**攻击方式**
- 拉普拉斯机制添加噪声的特点是**均值为0**
- 如果攻击者向数据库请求重复执行同一个查询语句，**将结果求和平均，就有极大的概率获得真实结果**

攻击者攻击成功概率表

ϵ	噪声绝对值的数据分布				多次查询添加噪声的平均值落在危险区间内的概率			
	90%	95%	99%	99.9%	100	1000	10000	10000
1	2.29	2.98	4.50	6.43	100.00	100.00	100.00	100.00
0.1	23.24	29.99	45.51	66.56	25.59	73.75	99.99	100.00
0.01	227.97	296.22	463.48	677.26	2.72	9.12	27.85	73.70

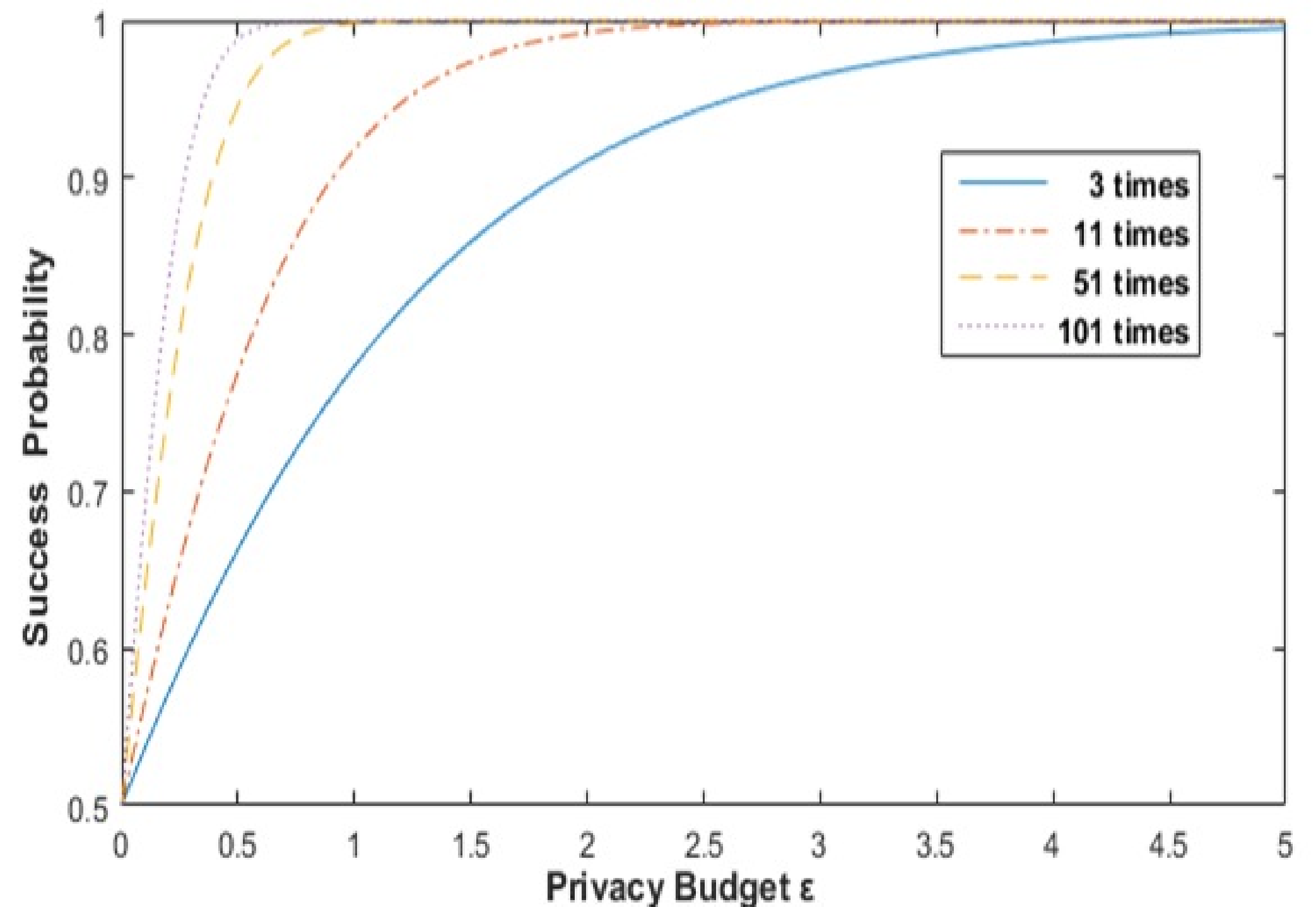
方案一：交互式发布

对抗重复推测攻击的差分隐私系统

安全算法

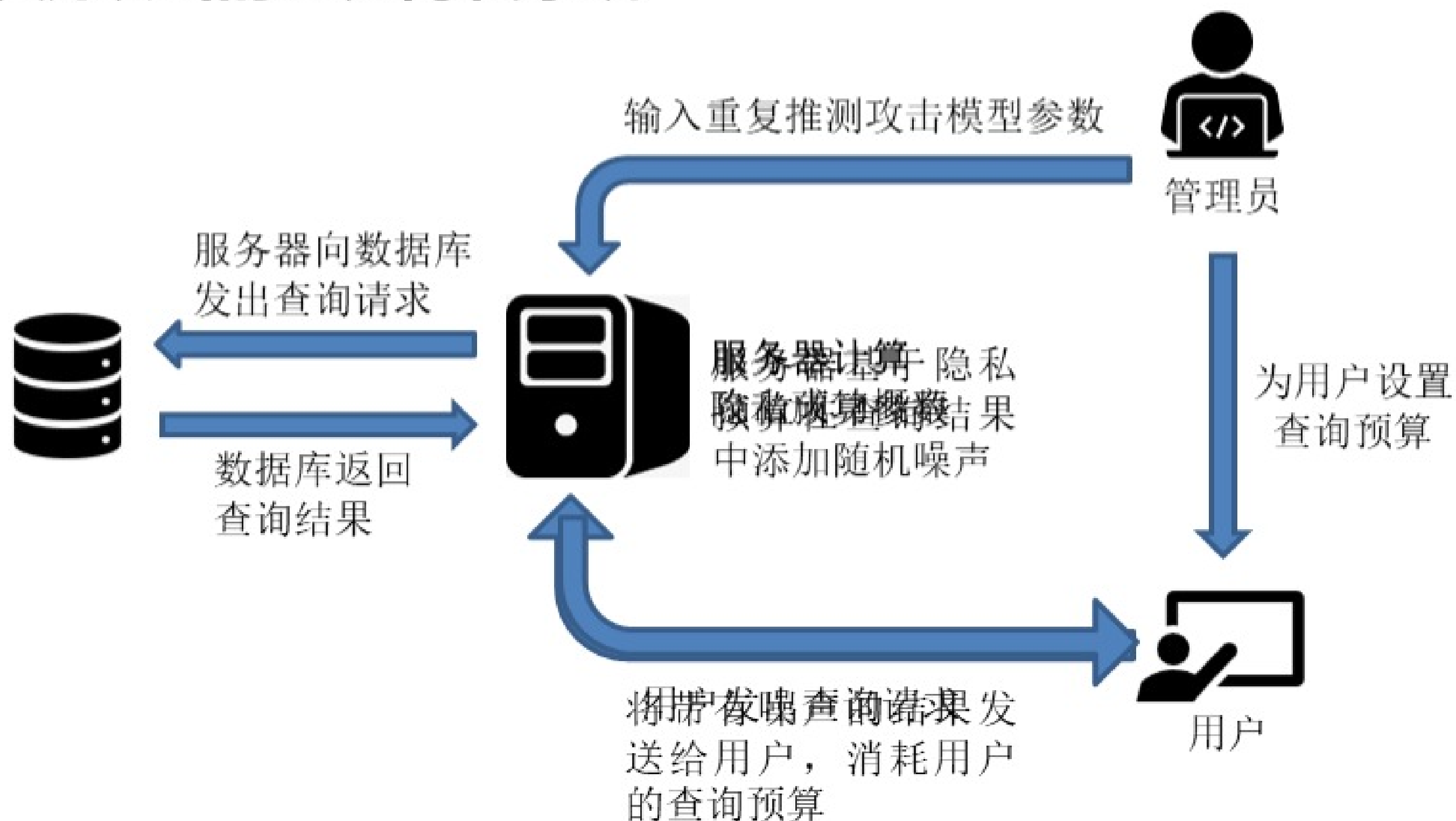
- 重复推测攻击成功概率与**查询次数**以及**隐私预算参数 ϵ** 有关。
- **通过提供目标限制的查询次数，计算安全的隐私预算参数 ϵ** ，抵御重复推测攻击，是一种动态适配性的机制。

重复推测攻击成功概率



方案一：交互式发布

对抗重复推测攻击的差分隐私系统



方案一：交互式发布

- 事实上，隐私预算 ϵ 刻画了一类查询任务的总体允许的隐私泄露程度。如果仅仅将 ϵ 作为生成拉普拉斯噪声的参数，多次查询很容易就实现隐私信息的推断。
- 要考虑保护多次查询的话，需要为每次查询进行预算分配：假定隐私预算为 ϵ ，允许的查询次数为 k ，则每次查询分配的预算为 $\frac{\epsilon}{k}$ ，这样才能达到 ϵ -差分隐私的目标。
- 因此，对于统计查询而言，如果在查询结果上进行反馈，则需要定义所能支持的次数，进而按上述方式对每次查询进行预算的分配。换句话说，这种添加噪音的方式，会使得每次查询都会消耗一定的隐私预算，直到隐私预算都被消耗干净，就再也不能起到保护的作用。

方案二：非交互式发布

- 不在结果上加噪音，而是数据上加噪音，产生加噪后的数据，以产生的加噪的数据集来响应查询。在这种情况下，我们对每条记录都根据设定的隐私预算产生相关的拉普拉斯噪声，并添加，进而生成合成的数据集。

```
void csv_analysis(char* path, double beta, long int seed){
    while(original_data[i].name)    {

        x = laplace_data(beta,&seed); //产生拉普拉斯噪音
        printf("Added noise:%f\t%s %d\t%f\n",x, original_data[i].name,
original_data[i].carrots+x); //当投入较少预算时，可能会出现负数
        if(original_data[i].carrots+x>=55)        {
            sum++;        }
        i++;    }
    printf("Animals which carrots cost > 55 (Under DP): %d\n",sum); //输出加噪后
的数据集中，每日食用胡萝卜大于55的动物个数
}}
```


方案二：非交互式发布

- 为了刻画信息泄露的情况，设计一个相邻数据集zoo_nb.csv（去掉了 “Dugeng”这一项数据），来对比演示加入不同规模的噪音对统计结果的影响。在主函数中，指定全局敏感度为1，以10和0.1作为两个隐私预算，进行对比比较：

```
int sen = 1;    //对于一个单属性的数据集，其敏感度为1
double eps[]={10, 0.1}; //指定两个隐私预算，分别代表极大，极小
while(i<2)    {
    double beta = sen / eps[i]; //拉普拉斯机制下，实际公式的算子
    beta为敏感度/预算
    seed = rand()%10000+10000; //随机种子产生
    csv_analysis("./zoo.csv",beta,seed); //先调用原始数据集
    seed = rand()%10000+10000; //随机种子更新
    csv_analysis("./zoo_nb.csv",beta,seed); //再调用相邻数据集
    i++;
}
```


方案二：非交互式发布

□ 输入较大的隐私预算（10），观察生成的噪音和加噪后的数据与原始数据的差别：

```
Under privacy budget 10.000000, sanitized original data with animal name and laplace noise:
Animals which carrots cost > 55 (original): 90
Added noise:0.975233      Aardvark      1.975233
Added noise:0.059110      Albatross     88.059110
Added noise:1.146398      Alligator     36.146398
Added noise:1.003976      Alpaca        100.003976
Added noise:0.168818      Ant           69.168818
Added noise:0.057197      Anteater      14.057197
Added noise:0.922177      Antelope      77.922177
Added noise:-0.012914     Ape           52.987086
Added noise:-0.001565     Armadillo     93.998435
Added noise:0.911597      Baboon        67.911597
Added noise:0.038145      Badger        92.038145
Added noise:0.828769      Barracuda     87.828769
Added noise:-0.077417     Bat           69.922583
Added noise:1.027173      Bear          32.027173
Added noise:-0.049532     Beaver        13.950468
Added noise:0.100204      Bee           14.100204
Added noise:0.060420      Bison         61.060420
Added noise:0.423178      Boar          57.423178
Added noise:0.812943      Buffalo       68.812943
Added noise:0.666850      Butterfly     13.666850
```

□ 添加的噪音均小于1或略大于1

□ 加噪后数据集对该查询的响应

仍与数据集的变化一致，均为

89，体现出了Dugong离开数

据集造成的差异，不能有效抵

御对该查询的差分攻击

方案二：非交互式发布

□ 输入较小的隐私预算（0.1），观察生成的噪音和加噪后的数据与原始数据的差别：

```
Under privacy budget 0.100000, sanitized original data with animal name and laplace noise:
```

```
Animals which carrots cost > 55 (original): 90
```

Added noise:1.093426	Aardvark	2.093426
Added noise:1.553708	Albatross	89.553708
Added noise:-0.301600	Alligator	34.698400
Added noise:2.885617	Alpaca	101.885617
Added noise:2.400248	Ant	71.400248
Added noise:-7.071718	Anteater	6.928282
Added noise:16.816570	Antelope	93.816570
Added noise:0.086264	Ape	53.086264
Added noise:-1.384364	Armadillo	92.615636
Added noise:-0.071994	Baboon	66.928006
Added noise:1.996502	Badger	93.996502
Added noise:0.572451	Barracuda	87.572451
Added noise:-0.678219	Bat	69.321781
Added noise:4.313268	Bear	35.313268
Added noise:3.524837	Beaver	17.524837
Added noise:4.672717	Bee	18.672717
Added noise:5.634807	Bison	66.634807
Added noise:5.257841	Boar	62.257841
Added noise:13.826595	Buffalo	81.826595
Added noise:6.462843	Butterfly	19.462843
Added noise:6.359006	Camel	27.359006
Added noise:0.854275	Caribou	38.854275
Added noise:3.405549	Cat	95.405549
Added noise:-0.548326	Caterpillar	38.451674

- 添加的噪音变大，统计查询变为95，加噪后的相邻数据集查询结果与加噪前相比变化巨大
- 投入较少的隐私预算时，虽然数据的可用性降低了，但是能够更好地抵御差分攻击的影响

直方图发布应用

直方图发布也是差分隐私数据发布的一类重要应用。

一个简单的医疗数据场景：假设其建立于某医疗数据集medicaldata.csv之上，其第一列数据为一特定年龄区间，第二列数据为该年龄区间患有某种疾病的人数。要发布的直方图即是以第一列数据为直方图的桶（Bucket）发布的统计数据。

A	B
20-30	405
30-40	436
40-50	421
50-60	457
60-70	463

交互式发布方案

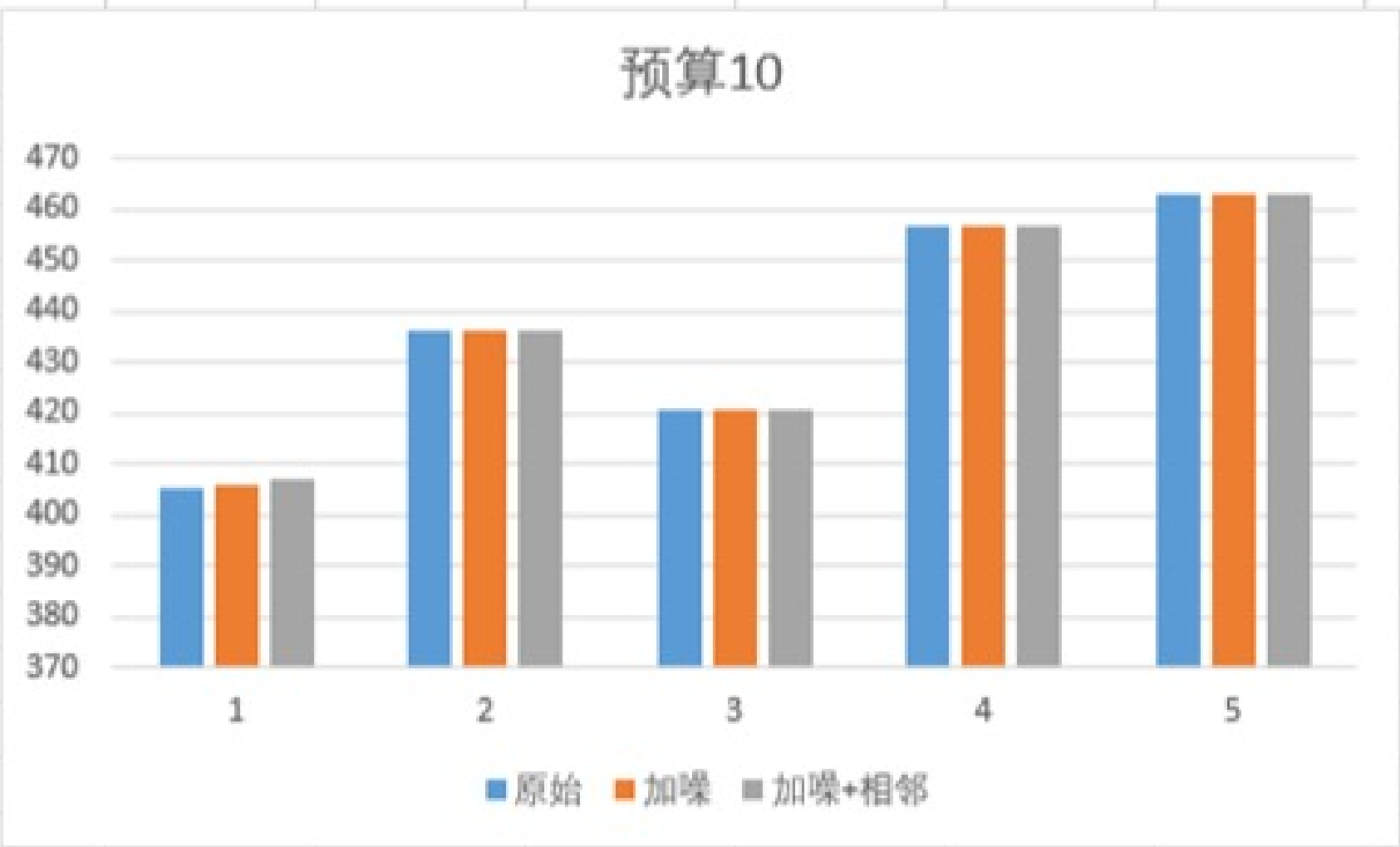
□ 直接在数据上加噪

```
void csv_analysis(char* path, double beta, long int seed){
    FILE *original_file = fopen(path,"r+"); //读取指定路径的数据集
    struct Histobuckets * original_data = NULL;
    original_data = hb_csv_parser(original_file);
    int sum=0,i=0;      double x = 0;
    while(original_data[i].bucket) {//循环为原始数据集内各桶数据生成拉普拉斯噪音
        x = laplace_data(beta,&seed); //产生拉普拉斯随机数
        printf("Added
noise:%f\t%s\t%f\n",x,original_data[i].bucket,original_data[i].count+x); //此处
        分别列出了每条具体添加的噪音和加噪的结果。当投入较少预算时，可能会出现负数
        i++;
    }
}
```

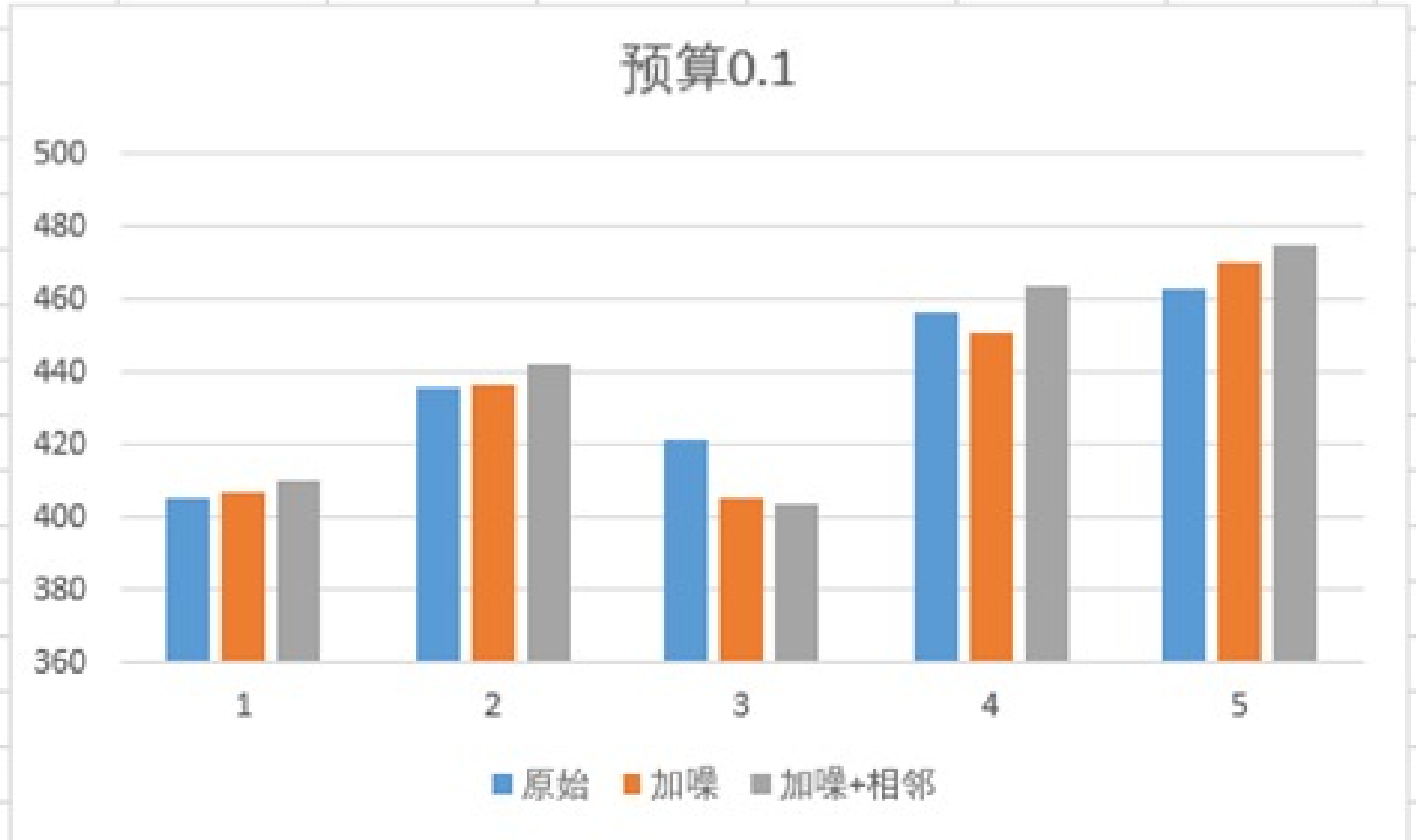

交互式发布方案

A	B
20-30	405
30-40	436
40-50	421
50-60	457
60-70	463

分区	原始	加噪	加噪+相邻
20	405	406	407
30	436	436	436
40	421	421	421
50	457	457	457
60	463	463	463



分区	原始	加噪	加噪+相邻
20	405	407	410
30	436	437	442
40	421	405	404
50	457	451	464
60	463	470	475



在交互式发布中，假定隐私预算为 ε ，允许的查询次数为 k ，则每次查询分配的预算为

- A ε
- B k
- C ε/k
- D εk

目 录

「01」

知识点一：概念和性质

「02」

知识点二：拉普拉斯机制

「03」

知识点三：差分隐私实践

「04」

知识点四：指数机制

「05」

知识点五：随机响应机制

1. 适用场景

对于一些特别的查询，直接对查询返回的结果直接加入随机噪音可能会造成较大的影响。

举例来说：将南瓜卖给Alice、Bob、Charlie，其中Alice和Bob最多能接受每个南瓜1元，而Charlie则可以接受每个南瓜3元。此时，我们想要定下价格，让我们受益最大。

如果我们定价为1元，这个价格在每个人的预算内，那我们能够获得3元（ $1+1+1$ ）；如果我们定价3元，那只有Charlie能买得起，那我们也可以获得3元的收益。此时我们可以发现，如果我们试图为最优定价加价，我们有两个最优定价可选，即1元和3元。

1. 适用场景

□ 为了保护我们的定价，可以加入噪音，但是当加入了一个噪音时，即使这个噪音极小，也会大幅度影响我们的收益。

当我们定价为1.01元时，只有Charlie仍然愿意购买，我们只能收入1.01元。

而当我们定价3.01元时，Charlie也不会购买，我们的收益会变成0元。

□ 这类查询(从1和3两个定价中选一个) 和非数值类查询的特点类似，其查询结果是离散的、不能变动的数据。

□ 对于这类查询（查询结果为实体对象，如一种选择或一个方案），我们通常使用指数机制来进行处理。

2. 工作原理

- 指数机制可以理解为对于某一问题的**隐私抽样算法**：对于一个一个问题 q ，存在一个打分函数 $q(D, r) \rightarrow R$ ，为数据集 D 的输出域 O 中的每一个可能的输出 r 进行打分，**指数机制会以打分函数为基础为输出 r 分配概率来对结果进行抽样。**
- 指数机制的目标就是降低数据集 D 中任意记录的改变对打分函数的影响，从而限制函数的输出导致数据集 D 中记录的隐私泄露。

2. 工作原理

定理5.5 指数机制 设随机算法 M 输入为数据集 D ，**抽样输出**为一实体对象 $r \in O$ ， $q(D, r)$ 为打分函数， Δq 代表打分函数 $q(D, r)$ 的灵敏度。当算法 M 满足

$$M(D, q) = \{r \mid \Pr[r \in O] \propto \exp\left(\frac{\varepsilon \times q(D, r)}{2\Delta q}\right)\}$$

提供 ε -差分隐私。

可以理解为， $M(D, q)$ 是从输出域 O 中抽样输出一个实体对象 r 的随机算法，抽样概率正比于 $\exp\left(\frac{\varepsilon \times q(D, r)}{2\Delta q}\right)$ 。

2. 工作原理

我们再给出一个指数机制的应用实例。**假如某学校要举办一场体育比赛，可供选择的项目来自集合{足球，排球，篮球，网球}，参与者们为此进行了投票，现要从中确定一个项目，并保证整个决策过程满足 ϵ - 差分隐私。**

对每一个参与者，他的投票选择都是个人隐私，不能泄露他喜欢哪种球类。诚如我们所知，差分攻击者可以通过两次输出的结果，来判断其中一个用户的喜好。**指数机制将依据计算得到的抽样概率进行抽样输出，抽样概率使得差分攻击者难以有绝对优势猜测出攻击者到底投了哪一个项目。以得票数量为打分函数，显然 $\Delta q = 1$ 。**

2. 工作原理

项目	打分函数 $\Delta q = 1$	概率		
		$\varepsilon = 0$	$\varepsilon = 0.1$	$\varepsilon = 1$
足球	30	0.25	0.424	0.924
排球	25	0.25	0.330	0.075
篮球	8	0.25	0.141	1.5e-05
网球	2	0.25	0.105	7.7e-07

当预算为0时，四个项目的概率被均分了，指数机制无论抽样得到哪个输出，差分攻击者都不能获得任何有用的信息。而在 ε 较大时，打分最高的选项被输出的概率被放大，反之当 ε 较小时，各个选项在打分上的差异被平抑，其输出概率随着 ε 的减小而趋于相等。

目 录

「01」

知识点一：概念和性质

「02」

知识点二：拉普拉斯机制

「03」

知识点三：差分隐私实践

「04」

知识点四：指数机制

「05」

知识点五：随机响应机制

1. 适用场景

- 前两种机制属于中心化差分隐私的范畴，即数据收集者是可信的，用户发送给数据收集者的数据一定是真实的，随机性的添加由数据收集者完成。**随机响应机制认为数据收集者并不可信，其随机性的添加在用户发送数据前就完成了，即数据收集者收集到的数据已经是被用户扰动过的。**
- 在现实生活中，各手机厂商或是各软件厂商的大数据分析就是类似的场景，我们并不能完全信任收集我们数据的厂商，所以我们需要随机响应机制来进行保护。**目前，谷歌、苹果、小米等大型厂商已经开始了对随机响应机制的使用。**

2. 工作原理

- 随机响应机制的主要思想是利用对敏感问题响应的不确定性对原始数据进行隐私保护，保证不同的单个记录之间的不可区分性，其随机性存在于用户向数据收集者提交数据之前。
- 随机响应技术主要包括**两个步骤**：**扰动性统计和校正**。
- **举例**：以询问用户是否患某种病症为例，假设有 n 个用户，其中**患病的真实比例为 π** ，但数据收集者并不知道。收集者希望对比例 π 进行统计，于是发起一个敏感问题的询问，即“你是否患病”，每个用户对该问题进行响应，用1表示“是”，0表示“否”，第 i 个用户给出答案 X_i ，但出于隐私性考虑，用户不直接响应真实结果，而是通过某种方式来随机决定是否输出这个结果。

2. 工作原理

- 这种随机性有多种实现方式。如本节此前所述，一种最简单的实现方式是用户可以**丢一枚硬币**，并根据一次或者多次投掷，并界定这些投掷结果的排列组合对应的输出来实现随机化，如两次投掷时，“正正”输出1，“正反”输出0等。
- 还有一种类似的例子是采用**掷骰子的方式**来实现随机化。对于是否输出自己真实的疾病情况的调查，我们要求用户掷一个骰子，若结果为1则再掷一次，结果为4-6则返回0。第二次投掷时，若其结果为1则返回1，其他情况返回0。显而易见，这种二次掷骰方法输出1的概率为 $1/6 * 1/6 = 1/36$ 。
- **通过迭代掷骰和设计输出规则，我们可以实现满足任意概率的输出方式，满足任意 ϵ 定义。**

2. 工作原理

- 从更高层次来看，无论是使用多次掷硬币，还是多次掷骰，其均等同于借助于一枚非均匀的硬币来给出答案，而其**正面向上的概率为 p** ，**反面向上的概率为 $1 - p$** 。抛出该硬币，若正面向上则回答真实答案，反面向上则回答相反的答案。在扰动性统计过程中，利用上述方法对 n 个用户的回答进行统计，可以得到患者人数的统计值。
- 假设统计结果中，**回答“是”的人数为 n_1** ，**则回答“否”的人数为 $n - n_1$** 。显然按照上述统计，回答“是”和“否”的用户比例分别为

$$\Pr[X_i = 1] = \pi p + (1 - \pi)(1 - p)$$

$$\Pr[X_i = 0] = (1 - \pi)p + \pi(1 - p)$$

2. 工作原理

显然，由于掷硬币的不均匀性影响，我们此时获得的统计比例与我们期望得到的真实统计比例 π 与 $1 - \pi$ 并不相等，**并非是无偏统计。因此，我们需要对统计结果进行校正。**

构建以下似然函数：

$$L = (\pi p + (1 - \pi)(1 - p))^{n_1} ((1 - \pi)p + \pi(1 - p))^{n - n_1}$$

得到 π 的极大似然估计：

$$\hat{\pi} = \frac{p - 1}{2p - 1} + \frac{n_1}{(2p - 1)n}$$

2. 工作原理

以下关于 $\hat{\pi}$ 的数学期望保证其为真实分布 π 的无偏估计

$$E(\hat{\pi}) = \frac{1}{2p-1} \left(p-1 + \frac{1}{n} \sum_{i=1}^n X_i \right) = \frac{1}{2p-1} (p-1 + \pi p + (1-\pi)(1-p)) = \pi$$

由此可以得到校正后的统计值，**患病人数估计值为** $N = \hat{\pi} \times n = \frac{p-1}{2p-1} n + \frac{n_1}{2p-1}$ 。

综上，根据总人数 n 、回答患病的人数 n_1 和扰动概率，即可得到真实患病人数的统计值。

3. 举例

以性别的频数统计为例，我们来验证一下上面的推导。

原始数据：

——男：6，女：4

概率：

——0.8概率上报真实性别，0.2的概率上报假的性别

统计结果期望：

——男： $6*0.8+4*0.2=5.6$

——女： $6*0.2+4*0.8=4.4$

校正：

——男： $[10* (0.8-1) + 5.6]/ (0.8*2 - 1) = 6$

——女： $[10* (0.8-1) + 4.4]/ (0.8*2 - 1) = 4$

从统计学上，校正后的频数统计结果的期望是和真实值一致的。

4. 形式化定义

定理5.7 (二值) 随机响应机制 设随机算法 M 输入为记录 $x \in \{0,1\}$, 输出为 $y \in \{0,1\}$, 随机算法 M 会输出一个伯努利随机变量 $M(x)$, 其中:

$$\Pr[M(x) = x] = \frac{e^\epsilon}{e^\epsilon + 1}$$

$$\Pr[M(x) = 1 - x] = \frac{1}{e^\epsilon + 1}$$

则算法满足 ϵ -本地差分隐私。

4. 形式化定义

因为

$$\frac{Pr[M(x)=x]}{Pr[M(x)=1-x]} = e^{\varepsilon}, \quad Pr[M(x) = x] = p$$

$$Pr[M(x) = x] + Pr[M(x) = 1 - x] = 1,$$

为满足 ε -本地差分隐私, $\varepsilon = \ln \frac{p}{1-p}$ 。

在确定隐私预算 ε 之后, 就可以确定 p 的取值, 进而可以明确随机响应执行的过程。

随机响应机制中，校正的作用是

- ☐ A 解决扰动性带来的噪音问题，实现结果的纠偏
- ☐ B 添加噪音，达到差分隐私的效果

谢 谢

